

RDBMS Engine

Application (Embedded SQL)

Query Processor Transaction Processor

**Query Optimizer
and Executor**

Indexes

Memory Manager

(I) Concurrency Control

(A/D) Recovery

Operating System

Hardware

[Processors, Memory, Disks]

LRU-K MEMORY MANAGER

E0 261

Jayant Haritsa
Indian Institute of Science

LRU

- Basic Concept
 - When a new buffer page is needed, the buffer pool manager drops the page from buffer that has not been accessed for the longest time.
- Why LRU?
 - Logical and intuitive
 - Easy to implement
- Why not LRU?
 - Does not understand reference patterns!

Problem Case I

- A multi-user database application, which references **randomly** chosen customer records through a clustered B-tree indexed key, **CUST-ID**, to retrieve desired information.
- If pattern of access is: I1, **R1**, I2, **R2**, I3, **R3**,
- LRU may drop the index pages, which are likely to be referenced again, causing increased I/O.

Problem Case II

- A multi-process database application with good “locality” of shared page reference.
- 5000 buffered pages out of 1 million disk pages get 95% of the references by concurrent processes.
- **Sequential scans** will replace commonly referenced pages in buffer with pages unlikely to be referenced again
 - common complaint in commercial situations



Basic Limitation

- LRU decides what page to drop from buffer based on too little information, **limiting itself to only the time of last reference.**
- LRU is unable to **differentiate** between pages that have markedly different frequency of references.
- Results in increased I/O.
- In general, **bad for database applications.**



PRIOR LITERATURE

- Solutions fall into two categories
 - Page Pool tuning
 - DBA constructs **page pools**, separating different reference patterns into different buffer pools.
 - e.g. index pool separate from data pool
 - Supported by some commercial database systems, but requires significant **manual** effort, and not conducive to dynamic hot-spot intra-pool environments
 - Query Plan analysis
 - Query optimizer should provide **hints** about the usage pattern of a query plan.
 - e.g. DBMIN [VLDB85] uses **MRU** for Looping Sequential references
 - In multi-user situations, query optimizer plans may **overlap** in complicated ways.
- Need for general light-weight adaptive page replacement policy!



LRU-K algorithm

- Maintains history of last K references to page.
- Discriminates between page sets with different degrees of reference frequency (e.g., index pages vs. data pages).
- Adapts to evolving access patterns.
- Self-reliant, does not need any external hints.
- Simple with little bookkeeping overheads.
- Small K for dynamic, large K for stable access patterns
 - $K=2$ is good compromise!

INDEPENDENT REFERENCE MODEL

- Independent Reference Model is a conceptual model used in analysis of storage systems – disk drives, caches, etc.
- References to stored objects are independent random variables.



IRM

- Set $N = \{1, 2, \dots, n\}$ of disk pages.
- Succession of references to these pages specified by reference string: $r_1, r_2, \dots, r_t$ where $r_t = \text{page } p$ ($p \in N$)
- At instant t , we assume that each disk page p has a well-defined probability, b_p , to be next referenced page.

$$\text{Prob}(r_{t+1} = p) = b_p \quad \forall p \in N.$$

- We assume that b_p is independent of t .

IRM

- Set $N = \{1, 2, \dots, n\}$ of disk pages.
- Succession of references to these pages specified by reference string: $r_1, r_2, \dots, r_t$ where $r_t = \text{page } p$ ($p \in N$)
- At instant t , we assume that each disk page p has a well-defined probability, b_p , to be next referenced page.

$$\text{Prob}(r_{t+1} = p) = b_p \quad \forall p \in N.$$

- We assume that b_p is independent of t .

Ao ALGORITHM

- Let Ao denote the buffering algorithm that replaces the buffered page p in memory whose expected value l_p is the maximum, i.e., the page for which b_p is smallest.
- Algorithm Ao is optimal under the independent reference assumption (Coffman/Denning1973)
- LRU-K gives very close results to the optimal algorithm, even at $K=2$ itself!

BACKWARD K-DISTANCE

Given a reference string known up to time t , $r_1, r_2, \dots, r_t$, the backward K-distance $b_t(p, K)$ is the distance backward to the K^{th} most recent reference to the page p :

$$\begin{aligned} b_t(p, K) &= x, && \text{if } r_{t-x} \text{ has the value } p \text{ and there have been} \\ &&& \text{exactly } K-1 \text{ other values } i \text{ with} \\ &&& t-x < i \leq t, \text{ where } r_i = p, \\ &= \infty, && \text{if } p \text{ does not appear at least } K \text{ times in} \\ &&& r_1, r_2, \dots, r_t \end{aligned}$$

- Set $N = \{1, 2, \dots, n\}$ of disk pages.
- Succession of references to these pages specified by reference string: $r_1, r_2, \dots, r_t$ where $r_t = \text{page } p$ ($p \in N$)

- At instant t , we assume that each disk page p has a

EXAMPLE (K=2)

- Consider reference string:

t: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

p: 7, 5, 6, 2, 9, 1, 8, 7, 6, 8, 1

x: $\infty, \infty, \infty, \infty, \infty, \infty, \infty, 7, 6, 3, 5$

- LRU-2

- $b_{10}(p=1, k=2) = 5$

- $b_{10}(p=6, k=2) = 8$

- $b_{10}(p=9, k=2) = \infty$

- ...

- $b_{10}(p=5, k=2) = \infty$ [oldest infinity]

If buffer victim required, drop Page 5

BACKWARD K-DISTANCE

- Ability to make decisions by measurement of actual interarrival between references, rather than LRU approach of estimating simply by a lower bound of time to most recent reference
- Problems:
 - Early page replacement
 - Retained information

1. EARLY PAGE REPLACEMENT

- Four ways in which a pair of references might access same disk page :
 - 1) Intra-Transaction: Read record in page; Update record
 - 2) Transaction-Retry: Access page; Abort Xsaction; Reaccess page
 - 3) Intra-Process: Access Initial Records on page; Commit; Access Later Records in page.
 - 4) Inter-Process: Independent access by multiple transactions

First 3 are not genuinely independent references but artifacts of database processing, and are likely to occur in short span of time. Whereas 4 is what we want to capture and is likely to have longer time spans.

EARLY PAGE REPLACEMENT (contd)

- Interarrival time should be calculated based on non-correlated access pairs!
- Each successive access by the same process within a **time-out period (Correlated Reference Period)** is assumed to be correlated, and collapsed into same reference.
 - Correlated Reference Period: 5 seconds (DBA tuning)

2. RETAINED INFORMATION

- Set $N = \{1, 2, \dots, n\}$ of disk pages.
- Succession of references to these pages specified by reference string: $r_1, r_2, \dots, r_t$ where $r_t = \text{page } p$ ($p \in N$)
- At instant t , we assume that each disk page p has a well-defined probability, b_p , to be next referenced page.

$$\text{Prob}(r_{t+1} = p) = b_p \quad \forall p \in N.$$

- We assume that b_p is independent of t .

DATA STRUCTURES : LRU-K

- **HIST(p)**
 - Denotes history control block of page p
 - Contains times of 2 most recent uncorrelated references to page p
 - $\text{HIST}(p,1)$, $\text{HIST}(p,2)$ denote time of last and second-last (if exist) references to page p
- **LAST(p)**
 - Denotes the time of the most recent reference to page p
- Memory resident – not stored in page header because of disk I/Os during eviction of page (need to write due to metadata change, not data change)
- Maintained for all pages with Backward-K < RIP

PSEUDO CODE: LRU-K

```
Procedure to be invoked upon reference to page p at time t:

if p is already in the buffer
then /* update history information of p */
  if t - LAST(p) > Correlated_Reference_Period
  then /* a new, uncorrelated reference */
    correl_period_of_refd_page := LAST(p) - HIST(p,1)
    for i := 2 to K do
      HIST(p,i) := HIST(p,i-1) +
        correl_period_of_refd_page
    od
    HIST(p,1) := t
    LAST(p) := t
  else /* a correlated reference */
    LAST(p) := t
  fi
else /* select replacement victim */
  min := t
  for all pages q in the buffer do
    if t - LAST(q) > Correlated_Reference_Period
      /* if eligible for replacement */
      and HIST(q,K) < min
      /* and max Backward K-distance so far */
```

```
    then
      victim := q
      min := HIST(q,K)
    fi
  od
  if victim is dirty then
    write victim back into the database fi
  /* now fetch the referenced page */
  fetch p into the buffer frame previously held by victim
  if HIST(p) does not exist
  then /* initialize history control block */
    allocate HIST(p)
    for i := 2 to K do HIST(p,i) := 0 od
  else
    for i := 2 to K do HIST(p,i) := HIST(p,i-1) od
  fi
  HIST(p,1) := t
  LAST(p) := t
fi
```

TWO-POOL EXPERIMENT

140/60

- Pool 1 with $N_1=100$ pages
Pool 2 with $N_2=10000$ pages
Vary Buffer pool size B
- In this two-pool experiment,
alternating uniformly
distributed references:
 $I_1, R_1, I_2, R_2, I_3, R_3 \dots$

B	LRU-1	LRU-2	LRU-3	A_0	$B(1)/B(2)$
60	0.14	0.291	0.300	0.300	2.3
80	0.18	0.382	0.400	0.400	2.6
100	0.22	0.459	0.495	0.500	3.0
120	0.26	0.496	0.501	0.501	3.3
140	0.29	0.502	0.502	0.502	3.2
160	0.32	0.503	0.503	0.503	2.8
180	0.34	0.504	0.504	0.504	2.5
200	0.37	0.505	0.505	0.505	2.3
250	0.42	0.508	0.508	0.508	2.2
300	0.45	0.510	0.510	0.510	2.0
350	0.48	0.513	0.513	0.513	1.9
400	0.49	0.515	0.515	0.515	1.9
450	0.50	0.517	0.518	0.518	1.8

Table 4.1. Simulation results of the two pool experiment, with disk page pools of $N_1 = 100$ pages and $N_2 = 10,000$ pages. The first column shows the buffer size B . The second through fifth columns show the hit ratios of LRU-1, LRU-2, LRU-3, and A_0 . The last column shows the equi-effective buffer size ratio $B(1)/B(2)$ of LRU-1 vs. LRU-2.

ZIPFIAN RANDOM ACCESS

- A single pool of pages with skewed random access. References to $N=1000$ pages with a Zipfian (80-20) distribution of reference frequencies.

B	LRU-1	LRU-2	A_0	$B(1)/B(2)$
40	0.53	0.61	0.640	2.0
60	0.57	0.65	0.677	2.2
80	0.61	0.67	0.705	2.1
100	0.63	0.68	0.727	1.6
120	0.64	0.71	0.745	1.5
140	0.67	0.72	0.761	1.4
160	0.70	0.74	0.776	1.5
180	0.71	0.73	0.788	1.2
200	0.72	0.76	0.825	1.3
300	0.78	0.80	0.846	1.1
500	0.87	0.87	0.908	1.0

Table 4.2. Simulation results on buffer cache hit ratios for random access with Zipfian 80-20 distribution to a disk page pool of $N=1000$ pages.

OLTP TRACE

- 1-hour page reference trace of production OLTP system of large bank.
- Trace contains ~0.5M page references to a 20 GB database.

B	LRU-1	LRU-2	LFU	B(1)/B(2)
100	0.005	0.07	0.07	4.5
200	0.01	0.15	0.11	3.25
300	0.02	0.20	0.15	3.0
400	0.06	0.23	0.17	2.75
500	0.09	0.24	0.19	2.4
600	0.13	0.25	0.20	2.16
800	0.18	0.28	0.23	1.9
1000	0.22	0.29	0.25	1.6
1200	0.24	0.31	0.27	1.66
1400	0.26	0.33	0.30	1.5
1600	0.29	0.34	0.31	1.5
2000	0.31	0.36	0.33	1.3
3000	0.38	0.40	0.39	1.1
5000	0.46	0.47	0.44	1.05

Table 4.3. Simulation results on buffer cache hit ratios using an OLTP trace.

SUMMARY

- LRU – does not maintain adequate history
- LFU – maintains unweighted history
- LRU-K – maintains weighted (recent) history
- History maintained for “non-resident” pages



END LRU-K

E0 261

OPTIMALITY OF LRU-K

- Set $N = \{1, 2, \dots, n\}$ of disk pages.
- Succession of references to these pages specified by reference string: $r_1, r_2, \dots, r_t$ where $r_t = \text{page } p$ ($p \in N$)
- At instant t , we assume that each disk page p has a well-defined probability, b_p , to be next referenced page.

$$\text{Prob}(r_{t+1} = p) = b_p \quad \forall p \in N.$$

- We assume that b_p is independent of t .